

השלב הבא: מסד נתונים ו-Backend

המסמך הזה נועד לרכז ולסכם את מה שכל אחת משלוש הקבוצות הגישה, ולסמן את הגרסה הסופית שממנה כולן ממשיכות. כך גם כל קבוצה יודעת מה הקבוצות האחרות עשו ובונות עליו — לא רק על מה שהגישה בעצמה.

עברתי על מה ששלוש הקבוצות הגישו. שלושתכן עשיתן עבודת ניתוח רצינית על המערכת כולה, בדיוק כמו שהתבקש בשבוע הלמידה. עכשיו — מהמסמכים לקוד.

סטאק Node.js + TypeScript + React מסד נתונים MongoDB עיקרון מרכזי קוד גנרי, לא משוכפל

תזכורת – מי אחראית על מה

כפי שנקבע בנוהל העבודה: כל קבוצה ניתחה השבוע את המערכת כולה, אבל האחריות על הבנייה בפועל מתחלקת כך:

קבוצה	תלמידות	אחריות
קבוצה א'	אושניסקי טובה, זלצמן יעל, טוויג שיר	משרה, מפ"ל, ותצוגת הסטטוס הכללי של כל התהליך (שלב 1-3)
קבוצה ב'	עסיס יעל, אמיתי תמר, רוזנר הדסה	מועמדים, ניקוד, שקלול, והחלטה סופית (שלב 4-6)
קבוצה ג'	בצלאל יהודית רבקה, ניסן רבקה, פרץ אסתר	הרשאות, יצירת מסמכים, ותשתית

מה שראיתי – שלושת המקורות

שלוש עבודות ניתוח שונות, כל אחת עם נקודות חוזק משלה. הנה תמצית:

Recruitment Management System

github.com/shirtwig/Recruitment-Management-System

- **Company** כישות נפרדת — חברת הגיוס אינה משתמש ואינה מועמד
- מועמד נפרד ממועמדות — אותו מועמד יכול להגיש למספר משרות
- **STATUS_LOG** — יומן ביקורת נפרד: כל פעולה שומרת סטטוס, מבצע ותאריך

הרעיון הכי חזק: הפרדת חברת הגיוס כישות משלה, ויומן ביקורת מוקדש.

NextHire

github.com/hadasa-r1/NextHire

- שלב (Stage) הוא ישות משלו, עם משקל ומכסה — לא רק שדה בתוך קריטריון
- הפרדה בין התקדמות מועמד בשלב (APPLICATION_STAGE) לבין הציון בכל קריטריון (CRITERION_RESULT)
- הדגמה ויזואלית מלאה של 22 מסכים: שיבוץ מראיינים הוגן (אקראי, בלי ניגוד עניינים), דירוג שנחשף רק אחרי נעילה, עתודה חכמה
- הרעיון הכי חזק: מפריד "שלב" מ"קריטריון" — מבנה שתומך בכל תהליך הערכה, לא רק בזה שכבר מכירים.

assessments – מפ"ל DevOps

DevOps הדגמת + [טבלאות והקשרים.xlsx](#)

- קריטריון עם סוג מפורש — **BOOLEAN** (תנאי סף) מול **SCORED** (משקל ב-%)
- מיווג מועמדות עם הצעת המחיר (תעריף שעות, קו"ח) לישות אחת — **CandidateBid**
- **TenderSummary** — שורת סיכום מוכנה לועדה: ציון איכות, ציון מחיר, ציון סופי, דירוג, זוכה
- הדגמה קונקרטית, שדה-אחר-שדה, על משרת DevOps אמיתית

הרעיון הכי חזק: פירוט ברמת שדה בודד, ותצוגת סיכום מוכנה בדיוק כמו שהועדה צריכה לראות.

המודל המאוחד – הכי טוב משלוש

לא בוחרים גרסה אחת — לוקחים את הרעיון החזק ביותר מכל מקור. זה המודל שעליו כולן בונות מעכשיו:

Authorization

User

משתמש (צוות פנימי)

מנהל מקצועי, רפרנט, ועדה, מראיין, מנהל מערכת. לא מועמדים ולא חברות. מקושר ל-**Authorization** במקום שדה תפקיד קבוע.

NextHire + RMS

הרשאה

פרופיל הרשאות: אילו פעולות (קריאה/כתיבה) מותרות על אילו משאבים. האדמין יוצר ומנהל את הפרופילים, וכל משתמש מקושר לאחד מהם.

מ-RMS + הבהרת המנחה

Company

חברת גיוס / ספק

מגישה מועמדים בשם מועמד.

RMS-מ

Candidate

מועמד

האדם עצמו, בלי קשר למשרה ספציפית.

משותף לשלושה

Position

משרה

כותרת, אשכול, רמה, תעריף מרבי, שעות, משך התקשרות, סטטוס. מקושרת ל-**JobCategory**, אבל עם **Criterion/Stage** משלה.

משותף, שדות מ-**assessments**

JobCategory

קטגוריית משרה (תבנית)

כגון "DevOps" או "PMO". מכיל תבנית ברירת מחדל של שלבים וקריטריונים, שמועקת לכל משרה חדשה מאותה קטגוריה.

נוסף בעקבות השאלה של המנחה

Stage

שלב הערכה

שלב בתוך המשרה (למשל: ראיון טלפוני), עם משקל ומכסת מועמדים שעוברים הלאה.

מ-NextHire

Criterion

קריטריון (הגדרה)

מועקת מ-**JobCategory** (אותו שם קריטריון לכל משרות DevOps), אבל עם ערך יעד ספציפי למשרה הזו (למשל: 6 חודשים למשרה גיוניור, 6 שנים למשרה בכירה). סוג: BOOLEAN (סף) או SCORED (יחס / ישיר).

מ-**assessments** + הבהרת המנחה

Application

מועמדות

מועמד + משרה יחד — כולל הצעת המחיר (תעריף שעות, קו"ח, שלב נוכחי, עמידה בסף).

מיזוג RMS + assessments

EvaluationScore

קריטריון מלא (ציון)

לקריטריונים כמותיים (כמו ותק): הערך בפועל של המועמד (למשל 5 חודשים), והציון מחושב כיחס מול ערך היעד של הקריטריון (5÷6). לקריטריונים איכותיים (כמו רושם מראיון): ציון ישיר שהמראיין נותן. תמיד + הערות.

Document

מסמך

מכתב תיחור, פרוטוקולים, גרסאות.

מ-NextHire + RMS

TenderSummary

מפ"ל (השקלול הסופי)

סיכום השקלול של כל ה-EvaluationScore של מועמדות: ציון איכות, ציון מחיר, ציון סופי, דירוג, זוכה. זה ה"מפ"ל" עצמו.

מ-assesments

StatusLog

יומן ביקורת

כל פעולה, מי ביצע, מתי.

מ-RMS

דוגמה – למה זה שקלול ולא ציון ישיר

קריטריון "ותק" קיים בכל משרות ה-DevOps, אבל `targetValue` שלו שונה לפי המשרה: 6 חודשים במשרה ג'וניור, 6 שנים במשרה בכירה. מועמד עם 5 חודשים ומועמד עם 5 שנים מקבלים אותו ציון $(5 \div 6 \approx 83\%)$ — כי זה יחס, לא מספר מוחלט. הם כמובן לא מתחרים זה בזה, כי כל אחד שייך למפ"ל של משרה אחרת.

מפת הקשרים בין האוספים

כדי שכולן ידעו איך הכל מתחבר: כל שדה שמצביע לאוסף אחר, במקום אחד.

אוסף	שדה	מצביע ל
Position	categoryId	JobCategory
Position	createdBy	User
Stage	positionId	Position
Criterion	stageId	Stage

אוסף	שדה	מצביע ל
Application	positionId	Position
Application	candidateId	Candidate
Application	companyId	Company
EvaluationScore	applicationId	Application
EvaluationScore	criterionId	Criterion
EvaluationScore	interviewerId	User
TenderSummary	applicationId	Application (ייחודי)
TenderSummary	committeeApproverId	User
User	authorizationId	Authorization
Authorization	createdBy	User
Document	relatedId	לפי relatedType (כל אוסף)
StatusLog	entityId	לפי entityType (כל אוסף)

מ-יחסי ל-MongoDB

שלוש הקבוצות עוברות לאותו מסד נתונים, כדי שאפשר יהיה לבנות שכבת קוד משותפת אחת שכולן משתמשות בה.

בחשיבה היחסית	ב-MongoDB
טבלה	Collection
שורה	Document
מפתח זר (foreign key)	שדה עם ObjectId שמצביע למסמך באוסף אחר

שליפה נפרדת לפי `ObjectId`, או שמירה כפולה של שדה נוח (לפי הצורך)

JOIN

המשימה: להקים אוסף (`collection`) ב-MongoDB לכל ישות מהמודל המאוחד למעלה, עם השדות שכבר זוהו בשלושת המקורות.

כלל מרכזי לכתיבת ה-Backend: קוד גנרי

אל תכתבו `addPosition`, `addCandidate`, `addCriterion` בנפרד — כל אחת מהן עושה בדיוק אותו דבר (מוסיפה מסמך לאוסף). כותבים את זה פעם אחת, גנרית, ומשתמשים בה בכל מקום.

```
// מחלקה אחת גנרית, עובדת על כל אוסף
class Repository<T> {
  constructor(private model: Model<T>) {}

  async add(data: Partial<T>): Promise<T> {
    return this.model.create(data);
  }
  async getAll(): Promise<T[]> {
    return this.model.find();
  }
  async getById(id: string): Promise<T | null> {
    return this.model.findById(id);
  }
  async update(id: string, data: Partial<T>): Promise<T | null> {
    return this.model.findByIdAndUpdate(id, data, { new: true });
  }
  async remove(id: string): Promise<void> {
    await this.model.findByIdAndDelete(id);
  }
}

// כל קבוצה פשוט "מזינה" את הישות שלה למחלקה הגנרית
const positionRepository = new Repository<Position>(PositionModel);
const candidateRepository = new Repository<Candidate>(CandidateModel);
const criterionRepository = new Repository<Criterion>(CriterionModel);

// בחינם add / getAll / getById / update / remove עכשיו לכולן יש
await positionRepository.add({ title: "PMO" });
await candidateRepository.add({ name: "... " });
```

מתי כן כותבים פונקציה ייעודית? רק כשמשוהו באמת שונה מ-CRUD רגיל — למשל חישוב ה-TenderSummary המשוקלל, או קידום מועמד מהעתודה. לוגיקה עסקית ייחודית מקבלת פונקציה משלה, שמשתמשת ב-Repository הגנרי בפנים במקום לשכפל אותו.

איך זה נראה בפועל – מודל אחד לדוגמה

כל אוסף מוגדר באותה צורה: קודם Schema (השדות), אחר כך Model, ואז Repository סביבו. הנה Position כדוגמה — כל קבוצה חוזרת על אותו תבנית לאוספים שלה.

```
// position.model.ts
const positionSchema = new Schema<Position>({
  title: { type: String, required: true },
  clusterCode: String,
  roleCode: String,
  level: { type: String, enum: ["LEVEL_A", "LEVEL_C", "LEVEL_D"] },
  description: String,
  monthlyHours: Number,
  maxHourlyRate: Number,
  durationMonths: Number,
  status: { type: String, enum: ["DRAFT", "IN_EVALUATION", "APPROVED_FOR_TENDER", "CLOSED"] },
  submissionDeadline: Date,
  createdBy: { type: Schema.Types.ObjectId, ref: "User" },
});

const PositionModel = model<Position>("Position", positionSchema);
export const positionRepository = new Repository<Position>(PositionModel);
```

מי אחראית על מה השבוע – פירוט מלא

לכל קבוצה: שם האוסף (= שם ה-Model) והשדות המרכזיים שהוא צריך להכיל.

קבוצה א' – Position • JobCategory • Stage • Criterion • Company

• Position title, categoryId, clusterCode, roleCode, level, description, monthlyHours, maxHourlyRate, durationMonths, status, submissionDeadline, createdBy

- positionId, name, order, weightPercent, quota :**Stage**
- stageId, name, type (**BOOLEAN / SCORED**), scoringMethod (**RATIO / DIRECT**), :**Criterion**
- targetValue, weightPercent, maxScore, descriptionGuide
- name, description, stageTemplates :**JobCategory** (מערך מוטמע של שלבים + קריטריונים לדוגמה)
- name, companyIdNumber, contactEmail :**Company**

קבוצה ב' – **Candidate • Application • EvaluationScore • TenderSummary**

- fullName, idNumber, phone, email :**Candidate**
- positionId, candidateId, companyId, hourlyRateBid, resumeUrl, :**Application**
- passedThreshold, currentStage, rejectionReason
- applicationId, criterionId, interviewerId, actualValue, computedScore, :**EvaluationScore**
- notes, evaluatedAt
- totalQualityScore, priceScore, finalWeightedScore, applicationId (יחודי), :**TenderSummary**
- rankPosition, isWinner, committeeApproverId, awardLetterUrl

קבוצה ג' – **User • Authorization • Document • StatusLog**

- email, name, authorizationId, isActive :**User**
- name :**Authorization** (למשל "מנהל מקצועי"), permissions (מערך מוטמע של {resource, actions}), createdBy
- relatedType, relatedId, url, version, uploadedAt :**Document**
- entityType, entityId, action, performedBy, timestamp :**StatusLog**

דוגמה ל-Authorization

פרופיל "מנהל מקצועי": { resource: "Position", actions: ["READ","WRITE"] } .פרופיל "מראיין": { resource: "EvaluationScore", actions: ["WRITE"] } בלבד — יכול להזין ציונים, אבל לא לקרוא נתונים אחרים. האדמין (SYSTEM_ADMIN) יוצר ומעדכן את הפרופילים האלה; כל User מקושר לפרופיל אחד.

